Biketrip Advisor — Multi-Agent LLM Showcase

Projektbeschreibung

Biketrip Advisor ist eine Full-Stack-Anwendung, die demonstriert, wie mehrere spezialisierte LLM-Agenten in einer Pipeline orchestriert werden können. Die Anwendung erstellt personalisierte Radtourenpläne, indem sie 4 verschiedene Ollama-Modelle sequentiell einsetzt — jedes mit einer eigenen Rolle, eigenem System-Prompt und optimierter Temperatur. Ergänzt wird die Pipeline durch Retrieval-Augmented Generation (RAG) und Geo-Routing via externer APIs.

Pipeline-Architektur

```
Nutzeranfrage
    |
    ▼
┌─────────────────┐
│  Chat + RAG     │  mistral:7b (temp 0.7)
│  Kontextanrei-  │  → Holt relevante Routeninfos aus dem Embedding Store
│  cherung        │  → Kombiniert RAG-Kontext mit Nutzeranfrage
└─────────────────┘
         |
         ▼
┌─────────────────┐
│  Reasoning      │  deepseek-r1:8b (temp 0.2)
│  Analyse &      │  → Machbarkeit, Risiken, Budget
│  Bewertung      │  → Niedrige Temperatur für präzise Analyse
└─────────────────┘
         |
         ▼
┌─────────────────┐
│  Planning       │  qwen2.5:7b (temp 0.4)
│  Tagesplanung   │  → Tagesweise Etappen mit Orten und Distanzen
└─────────────────┘
         |
         ▼
┌─────────────────┐
│  Geo-Routing    │  Nominatim + OpenRouteService
│  (kein LLM)     │  → Geocoding der Orte → Routenberechnung → GeoJSON
└─────────────────┘
         |
         ▼
┌─────────────────┐
│  Language       │  llama3.1:8b (temp 0.8)
│  Report-        │  → Markdown-formatierter Reisebericht
│  Generierung    │  → Hohe Temperatur für lebendigen Stil
└─────────────────┘
```

Jeder Agent bekommt den Output des vorherigen Agenten als Input. Das Frontend zeigt die Ergebnisse live per Server-Sent Events (SSE) an — inklusive einer interaktiven Karte mit der berechneten Route.

Tech Stack

Komponente	Technologie	Version
Backend	Spring Boot + LangChain4j	3.4.4 / 0.36.2
Frontend	React + TypeScript + Vite	19 / 5.x
LLM Runtime	Ollama (selfhosted)	lokal oder remote
Embedding	AllMiniLmL6V2 (ONNX, in-process)	—
Karte	Leaflet + react-leaflet + OSM	1.9.4
Routing	OpenRouteService API	kostenloser Tier
Geocoding	OSM Nominatim	frei, 1 req/s
Container	Docker Compose (Multi-Stage)	—
Java	Eclipse Temurin	21
Node	Node.js	22

LLM-Einsatz im Detail

Bewusste Modellwahl pro Agent

Jeder der 4 Agenten verwendet ein anderes Ollama-Modell, das gezielt für seine Rolle ausgewählt wurde:

Agent	Modell	Temperatur	Begründung
Chat + RAG	mistral:7b	0.7	Guter Allrounder für kontextbezogene Konversation. Verarbeitet RAG-Kontext zuverlässig.
Reasoning	deepseek-r1:8b	0.2	Spezialisiert auf logisches Denken (Chain-of-Thought). Niedrige Temperatur für deterministische Analyse.
Planning	qwen2.5:7b	0.4	Starkes Strukturierungsvermögen für tabellarische Tagespläne. Moderate Temperatur für Kreativität innerhalb klarer Vorgaben.
Language	llama3.1:8b	0.8	Starke Sprachqualität für flüssige Markdown-Reports. Hohe Temperatur für lebendigen, natürlichen Schreibstil.

Runtime Model Swapping

Das Frontend bietet 4 Dropdown-Selektoren, über die jeder Agent zur Laufzeit ein anderes Modell erhalten kann — ohne Neustart. Das Backend baut das ChatLanguageModel dann dynamisch per

`OllamaChatModel.builder()`:

```
// BaseAgent.java – Model-Auflösung zur Laufzeit
private ChatLanguageModel resolveModel(String override) {
    if (override == null || override.isBlank()) {
        return defaultModel;
    }
    return LangChainConfig.buildModel(config.baseUrl(), override, temperature);
}
```

Die verfügbaren Modelle werden direkt von Ollama abgefragt (`GET /api/tags`) und nach Kategorien im Frontend angezeigt. So kann man ohne Code-Änderung verschiedene Modelle vergleichen — das Kernstück des Showcase.

Prompt Engineering

Jeder Agent hat einen eigenen deutschen System-Prompt (unter `resources/prompts/`), der Rolle, Erwartungen und Output-Format klar definiert. Die Prompts sind als externe `.txt`-Dateien ausgelagert und werden beim Start geladen — so sind sie ohne Neukompilierung anpassbar.

RAG-Integration (Retrieval-Augmented Generation)

Der Chat-Agent reichert die Nutzeranfrage mit relevantem Wissen an:

1. **Datenquellen**: 3 Textdateien mit realen Radrouten-Beschreibungen (Bodensee, Rhein, Schwarzwald)
2. **Embedding**: `AllMiniLmL6V2` läuft als ONNX-Modell direkt im Java-Prozess (23 MB) — kein externer Embedding-Service nötig
3. **Chunking**: `DocumentSplitters.recursive(500, 50)` — 500-Token-Chunks mit 50-Token-Überlappung
4. **Retrieval**: `EmbeddingStoreContentRetriever` mit maxResults=3 und minScore=0.5
5. **Augmentierung**: Gefundene Segmente werden strukturiert in den User-Prompt eingebettet

Der RAG-Ansatz zeigt, wie sich LLM-Antworten mit domänenspezifischem Wissen anreichern lassen, ohne das Modell fine-tunen zu müssen.

Externe Datenanbindung: Geo-Routing als "Tool"

Das Tool-Konzept in LLM-Pipelines

Ein zentrales Pattern moderner LLM-Anwendungen ist der Einsatz von **Tools** — externen Services, die ein Agent aufrufen kann, um strukturierte Daten zu erzeugen, die das LLM allein nicht liefern könnte. Im Biketrip Advisor ist der `GeoRoutingService` ein solches Tool: Er nimmt den unstrukturierten Text des Planning-Agenten, extrahiert daraus Ortsnamen, löst sie zu Koordinaten auf und berechnet eine reale Fahrradroute mit Distanz und Geometrie.

Im Gegensatz zu einem klassischen LangChain4j-Tool (das über `@Tool`-Annotation direkt vom LLM aufgerufen wird) ist das Geo-Routing hier als **Pipeline-Stage zwischen zwei Agenten** implementiert. Das hat einen bewussten Grund: Das Tool soll nicht vom LLM entscheiden, *ob* es aufgerufen wird — es wird *immer* nach dem Planning-Schritt ausgeführt, weil die Route ein fester Bestandteil des Outputs ist.

Datenfluss: Von LLM-Text zu GeoJSON

```
Planning-Agent Output (unstrukturierter Text)
  |
  |  "Tag 1: Freiburg → Breisach (45 km)"
  |  "Tag 2: Breisach → Colmar (38 km)"
  |  "Tag 3: Colmar → Basel (52 km)"
  |
  ▼
┌──────────────────────────────────────┐
│  1. Location Extraction (Regex)      │
│                                      │
│  Pattern: Tag\s*\d+\s*:?\s*(.+?)     │
│           \s*[→\->]+\s*(.+?)         │
│                                      │
│  → ["Freiburg", "Breisach",          │
│     "Colmar", "Basel"]               │
│  (LinkedHashSet: dedupliziert,       │
│   Reihenfolge erhalten)              │
└──────────────┬───────────────────────┘
               |
               ▼
┌──────────────────────────────────────┐
│  2. Geocoding (OSM Nominatim)        │
│                                      │
│  GET /search?q=Freiburg&format=json  │
│  → { lat: 47.999, lon: 7.842 }       │
│                                      │
│  Rate Limit: 1100ms zwischen Calls   │
│  Fehlertoleranz: Einzelne Orte       │
│  werden bei Fehler übersprungen      │
└──────────────┬───────────────────────┘
               |
               ▼
┌──────────────────────────────────────┐
│  3. Routing (OpenRouteService)       │
│                                      │
│  POST /v2/directions/                │
│       cycling-regular/geojson        │
│  Body: { coordinates: [[lon,lat],..]}│
│                                      │
│  → GeoJSON FeatureCollection         │
│    mit LineString-Geometrie          │
│    und Gesamtdistanz                 |
```

```
└──────────────┬──────────────────────────┘
               │
               ▼
┌──────────────────────────────────────────┐
│  RouteResult                             │
│  ├─ waypoints: [{name, lat, lon,         │
│  │               dayNumber}, ...]        │
│  ├─ geojson: FeatureCollection           │
│  └─ totalDistanceKm: 135.0               │
└──────────────────────────────────────────┘
```

Einbettung in die Pipeline

Der `PipelineOrchestrator` ruft das Geo-Routing gezielt zwischen Planning und Language ein:

```
// PipelineOrchestrator.java — Geo-Routing als Pipeline-Stage
for (PipelineAgent agent : agents) {
    if (agent.getRole() == AgentRole.LANGUAGE && !steps.isEmpty()) {
        route = runGeoRouting(currentInput, onRouteReady);
    }
    AgentStepResult step = agent.process(currentInput,
overrides.get(agent.getRole().name()));
    currentInput = step.output();
}
```

Das Timing ist bewusst gewählt:

- **Nach Planning**: Erst jetzt existieren Ortsnamen im Text
- **Vor Language**: Die Route wird als eigenes SSE-Event (`route-ready`) an das Frontend gestreamt, sodass die Karte *vor* dem finalen Report erscheint — der Nutzer sieht sofort die Route, während der Language-Agent noch arbeitet

Verwendete externe APIs

API	Zweck	Authentifizierung	Rate Limit	Kosten
OSM Nominatim	Geocoding (Ortsname → Koordinaten)	Keine (User-Agent Header)	1 req/s	Kostenlos
OpenRouteService	Fahrradrouting (Koordinaten → GeoJSON)	API-Key (Header)	40 req/min	Kostenlos (Free Tier)

Graceful Degradation

Das Tool ist so implementiert, dass die Pipeline bei jedem Fehlerszenario weiterläuft:

Fehlerfall	Verhalten	Ergebnis
Kein ORS-API-Key	Routing wird übersprungen	Karte zeigt nur Wegpunkt-Marker
Nominatim findet Ort nicht	Einzelner Ort wird übersprungen	Route mit weniger Wegpunkten
ORS-API nicht erreichbar	Routing-Call schlägt fehl	Waypoints ohne Routenlinie
Keine Orte im Plan extrahierbar	Geo-Routing gibt `null` zurück	Keine Karte, Report wird trotzdem erstellt
Gesamter GeoRoutingService schlägt fehl	`PipelineOrchestrator` fängt Exception, loggt WARN	Pipeline läuft weiter ohne Route

Warum kein `@Tool` / Function Calling?

In vielen LLM-Frameworks (LangChain4j, OpenAI Function Calling) kann ein LLM selbst entscheiden, ob und wann es ein Tool aufruft. Hier wurde bewusst darauf verzichtet:

1. **Determinismus**: Die Route wird *immer* berechnet, nicht nur wenn das LLM es für nötig hält
2. **Zuverlässigkeit**: Kleinere Modelle (7B/8B) sind bei Tool-Calling weniger zuverlässig als größere
3. **Trennung**: Das Tool ist eine eigenständige Pipeline-Stage mit eigenem SSE-Event — sauberer als ein Tool-Call innerhalb eines Agenten
4. **Testbarkeit**: Der `GeoRoutingService` ist unabhängig von LLMs testbar (6 Unit-Tests mit gemockten REST-Clients)

Dieses Pattern — **deterministische Tool-Aufrufe zwischen Agenten statt LLM-gesteuertes Function Calling** — ist besonders bei kleineren Modellen ein pragmatischer Ansatz, der Zuverlässigkeit über Flexibilität stellt.

Testbarkeit

Strategie: LLMs sind isoliert testbar

Die Architektur wurde bewusst so gestaltet, dass LLM-Aufrufe in Unit-Tests vollständig mockbar sind. Jeder Agent bekommt sein `ChatLanguageModel` per Constructor Injection — in Tests wird es durch ein Mockito-Mock ersetzt.

Backend-Tests (7 Testklassen, 31+ Testfälle)

Testklasse	Typ	Fokus
`PipelineOrchestratorTest`	Unit	Sequenzierung der Agenten, Callback-Aufrufe, Geo-Routing-Fallback
`ChatAgentTest`	Unit	RAG-Kontextanreicherung, System-Prompt-Inhalt, Rollenverifikation
`ReasoningAgentTest`	Unit	Machbarkeitsanalyse-Prompt, Input/Output-Mapping
`PlanningAgentTest`	Unit	Tagesplan-Struktur im Prompt, Input-Durchreichung
`LanguageAgentTest`	Unit	Markdown-Anweisungen im Prompt
`GeoRoutingServiceTest`	Unit	Ortsextraktion, Deduplizierung, Markdown-Bereinigung
`AgentIntegrationTest`	Integration	Live-Ollama-Aufrufe (via `@Tag("integration")` vom normalen Build ausgeschlossen)

LLM-Mocking-Muster:

```
// Beispiel: ChatAgentTest — LLM-Antwort wird simuliert
@Mock ChatLanguageModel chatLlm;
@Mock ContentRetriever contentRetriever;

@Test
void shouldAugmentInputWithRagContext() {
    when(contentRetriever.retrieve(any()))
        .thenReturn(List.of(textSegment("Bodensee-Radweg: 270km")));
    when(chatLlm.generate(any(SystemMessage.class), any(UserMessage.class)))
        .thenReturn(Response.from(AiMessage.from("Empfehlung...")));

    AgentStepResult result = chatAgent.process("Radtour am Bodensee", null);
    // Verifiziere, dass RAG-Kontext im Prompt enthalten war
}
```

Integrationstests:

Separate `@Tag("integration")`-Tests laufen gegen eine echte Ollama-Instanz. Sie sind vom normalen `mvn test`-Lauf ausgeschlossen (Maven Surefire Config) und validieren die Prompt-Qualität (z.B. "Output enthält Markdown-Überschriften").

Frontend-Tests (2 Testdateien, Vitest + React Testing Library)

Testdatei	Fokus
`pipelineApi.test.ts`	SSE-Parsing (step-complete, route-ready, error), HTTP-Fehler (429, 500), chunked Streaming
`usePipeline.test.ts`	State-Übergänge (idle → running → complete), Step-Akkumulation, Error-Handling, Reset

Die SSE-Streaming-Logik ist besonders gut getestet: Es werden chunked Responses simuliert, bei denen Events über mehrere `ReadableStream`-Reads verteilt ankommen — ein realistisches Szenario.

Testbarkeits-Bewertung

Aspekt	Bewertung	Details
LLM-Isolation	Sehr gut	Jeder Agent ist per Constructor Injection mockbar. Kein globaler State.
Prompt-Verifikation	Gut	`ArgumentCaptor` prüft System- und User-Messages in Unit-Tests.
Pipeline-Sequenzierung	Gut	Orchestrator-Tests verifizieren Reihenfolge und Callbacks.
Integrationstests	Vorhanden	Live-Ollama-Tests existieren, sind aber vom CI-Build getrennt.
Frontend SSE-Parsing	Sehr gut	Realistische Chunked-Stream-Szenarien getestet.
E2E-Tests	Nicht vorhanden	Kein Cypress/Playwright — bei einem Showcase akzeptabel.

Architektur-Qualität

Clean Architecture Prinzipien

- **Separation of Concerns**: Jeder Agent ist eine eigene `@Component`-Klasse mit klar definierter Rolle. Die `BaseAgent`-Abstraktion eliminiert Duplikation (Model-Auflösung, Timing, Prompt-Loading).
- **Dependency Injection**: Alle LLMs werden als Spring Beans mit `@Qualifier` injiziert. Kein `new` in Geschäftslogik.
- **Interface-basiert**: `PipelineAgent`-Interface ermöglicht einfaches Hinzufügen neuer Agenten.
- **`@Order`-basierte Sequenzierung**: Die Pipeline-Reihenfolge wird deklarativ über Spring-Annotationen gesteuert — nicht hart kodiert.

- **Record-basierte DTOs**: `AgentStepResult`, `PipelineResult`, `OllamaModelsConfig` etc. sind immutable Java Records.

Bewusste Design-Entscheidungen

Entscheidung	Begründung
Programmatischer Model-Bau statt `@AiService`	Ermöglicht Runtime Model Swapping über Frontend-Dropdowns
In-Process ONNX-Embedding statt Ollama-Embedding	Kein separater Embedding-Service nötig, schneller, offline-fähig
SSE statt WebSocket	Unidirektional ausreichend, einfacher, kein Session-Management
Virtual Threads (Java 21)	Blockierende Ollama-Aufrufe ohne Thread-Pool-Erschöpfung
`sessionStorage` statt State-Library	Ausreichend für Single-Page-App, kein Redux/Zustand-Overhead

Frontend-Architektur

- **Custom Hook `usePipeline`**: Zentralisiert State-Management und SSE-Handling
- **SSE-Parsing**: Manuell über `fetch` + `ReadableStream` implementiert (nicht `EventSource`), für bessere Kontrolle über POST-Requests und Error-Handling
- **Komponenten-Hierarchie**: Klar getrennt in Input (Form + ModelSelector), Pipeline (StepProgress + AgentStepCards), Output (FinalReport + RouteMap)

Logging

Strategie: Strukturiertes Logging auf allen Ebenen

Das Logging ist über SLF4J/Logback durchgängig implementiert und deckt den gesamten Request-Lifecycle ab:

Schicht	Log-Level	Was wird geloggt?
`PipelineController`	INFO	Request-Eingang, Message-Länge, Gesamtdauer
`PipelineOrchestrator`	INFO/WARN	Pipeline-Start/-Ende, Stage-Anzahl, Geo-Routing-Fehler
`BaseAgent`	INFO/DEBUG/ERROR	Model, Temperatur, Input-Länge, Dauer, Output-Länge, Fehler mit Stack-Trace
`ChatAgent`	INFO/DEBUG	RAG-Abfrage, Anzahl gefundener Segmente
`GeoRoutingService`	INFO/DEBUG/WARN/ERROR	Ortsextraktion, Geocoding pro Ort, Routing-Distanz, API-Fehler
`BikeRouteDataSeeder`	INFO	Seeding-Start, Segmente pro Datei, Gesamtzahl
`LangChainConfig`	INFO	Model-Initialisierung mit Name, Temperatur, URL

Logging-Qualität

- **Durchgängig**: Jeder Agent loggt Start, Dauer und Output-Länge — ermöglicht Performance-Vergleich zwischen Modellen
- **Model-Override-Tracking**: Bei Runtime Model Swap wird explizit geloggt, welches Override-Modell statt des Defaults verwendet wird
- **Graceful Degradation**: Geo-Routing-Fehler werden als WARN geloggt, nicht als ERROR — weil die Pipeline trotzdem weiterläuft
- **DEBUG-Ebene**: `de.biketrip.advisor` und `dev.langchain4j` auf DEBUG konfiguriert — LangChain4j loggt die tatsächlichen HTTP-Calls zu Ollama

Beispiel-Log-Output einer Pipeline-Ausführung

```
INFO  ChatAgent: starting with model=mistral:7b, temperature=0.7, input length=156 chars
INFO  ChatAgent: retrieved 3 relevant segments from RAG store
INFO  ChatAgent: completed in 4523ms, output length=892 chars
INFO  ReasoningAgent: starting with model=deepseek-r1:8b, temperature=0.2, input length=892 chars
INFO  ReasoningAgent: completed in 6781ms, output length=1245 chars
INFO  PlanningAgent: starting with model=qwen2.5:7b, temperature=0.4, input length=1245 chars
INFO  PlanningAgent: completed in 5102ms, output length=2034 chars
INFO  GeoRoutingService: extracted 6 locations, geocoded 6/6, route distance=287.3km
INFO  LanguageAgent: starting with model=llama3.1:8b, temperature=0.8, input length=2034 chars
INFO  LanguageAgent: completed in 8945ms, output length=3567 chars
INFO  Pipeline completed: 4 stages in 25891ms
```

Containerisierung

Multi-Stage Docker Builds

Beide Services verwenden Multi-Stage-Builds, die Build-Tools von der Runtime trennen:

Backend (Eclipse Temurin 21 Alpine):

- Stage 1: Maven-Build mit `dependency:go-offline` für Layer-Caching
- Stage 2: Nur JRE-Alpine + fertige JAR (minimaler Footprint)

Frontend (Node 22 → Nginx Alpine):

- Stage 1: `npm ci` (reproduzierbar) + Vite-Build
- Stage 2: Nginx Alpine serviert statische Dateien

Nginx als Reverse Proxy

Die `nginx.conf` ist für den Anwendungsfall optimiert:

- **SSE-Streaming**: `proxy_buffering off`, `proxy_cache off`, `gzip off` für Events — damit werden LLM-Antworten in Echtzeit an den Browser gestreamt
- **SPA-Fallback**: `try_files` leitet alle Routen auf `index.html` um
- **Security Headers**: CSP, X-Frame-Options, X-Content-Type-Options, Referrer-Policy
- **Context Path**: Alles unter `/biketrip-advisor/` — sauber für Reverse-Proxy-Setups

Docker Compose

```
services:
  backend:
    build: ./backend
    environment:     # Alle Ollama-Config per .env
      - BIKETRIP_OLLAMA_BASE_URL=${BIKETRIP_OLLAMA_BASE_URL}
      - BIKETRIP_OLLAMA_CHAT_MODEL=${BIKETRIP_OLLAMA_CHAT_MODEL}
      # ...
    ports: ["8080:8080"]

  frontend:
    build: ./frontend
    ports: ["3000:80"]
    depends_on: [backend]
```

Ein `docker compose up --build` startet die komplette Anwendung. Ollama läuft extern (auf dem Host oder einem dedizierten LLM-Server).

Containerisierungs-Bewertung

Aspekt	Bewertung	Details
Image-Größe	Gut	Alpine-basiert, Multi-Stage eliminiert Build-Tools
Layer-Caching	Gut	`dependency:go-offline` und `npm ci` als separate Layer
Reproduzierbarkeit	Gut	`npm ci` (Lockfile), Maven Offline-Dependencies
Security	Gut	Nginx Security Headers, kein Root-User nötig
SSE-Kompatibilität	Sehr gut	Nginx explizit für Event-Streaming konfiguriert
Health Checks	Nicht vorhanden	Kein `HEALTHCHECK` in Dockerfiles, kein `healthcheck:` in Compose
Secrets-Management	Basis	`.env`-Datei, `.env.example` als Template — kein Vault/Secrets-Manager

---

Self-Hosting

Voraussetzungen

Die Anwendung ist für vollständiges Self-Hosting auf eigener Hardware ausgelegt. Es werden keine Cloud-Services oder bezahlten APIs benötigt (OpenRouteService ist optional und kostenlos).

Hardware (LLM-Server)

Der LLM-Host benötigt eine GPU mit mindestens 8 GB VRAM für die 7B/8B-Modelle. Die aktuelle Konfiguration:

- **CPU**: AMD Ryzen 7 9700X (8-Core)
- **GPU**: NVIDIA RTX 4060 Ti (16 GB VRAM)
- **RAM**: 96 GB DDR5

Software

Komponente	Minimum	Empfohlen
Docker + Compose	v20+	aktuell
Ollama	aktuell	aktuell
GPU-Treiber	CUDA-kompatibel	NVIDIA 550+

Setup in 3 Schritten

```
# 1. Ollama-Modelle herunterladen (auf dem LLM-Host)
ollama pull mistral deepseek-r1:8b qwen2.5:7b llama3.1:8b

# 2. .env konfigurieren
cp .env.example .env
# BIKETRIP_OLLAMA_BASE_URL auf Ollama-Host setzen

# 3. Starten
docker compose up --build
# → http://localhost:3000/biketrip-advisor/
```

Graceful Degradation

Die Anwendung degradiert sauber, wenn externe Services nicht verfügbar sind:

Service	Bei Ausfall
OpenRouteService API	Karte zeigt nur Wegpunkt-Marker, keine Routenlinie
Nominatim Geocoding	Einzelne Orte werden übersprungen, Rest wird angezeigt
RAG-Kontext (keine Treffer)	Agent arbeitet ohne Kontext weiter
Ollama	Pipeline bricht mit Fehler ab (wird im Frontend angezeigt)

Zusammenfassung

Kategorie	Stärken	Verbesserungspotenzial
LLM-Einsatz	4 verschiedene Modelle mit rollenspezifischer Temperatur, RAG, Runtime Model Swap	Streaming der Token-Generierung (aktuell nur Step-Ergebnis-Streaming)
Testbarkeit	LLMs vollständig mockbar, 31+ Tests, SSE-Parsing getestet	E2E-Tests (Cypress/Playwright), Test-Coverage-Reports
Architektur	Clean Separation, Interface-basierte Pipeline, Virtual Threads	Event-driven statt sequentiell für parallele Agenten
Logging	Durchgängig auf allen Ebenen, Dauer + Modell pro Agent	Structured Logging (JSON), Correlation-IDs über die Pipeline
Containerisierung	Multi-Stage, Alpine, SSE-optimiertes Nginx	Health Checks, non-root User, Image-Scanning
Self-Hosting	Komplett offline-fähig (bis auf optionales Geo-Routing), einfaches Setup	Automatisches Ollama-Model-Pulling, Setup-Script